

Simulation Documentation Report

Course: ITAI 3377

Assignment: A02 – Simulation Documentation

Student Name: Kaire Campomanes

Title

Training and Deploying a Simple CNN Model Using Google Colab and TensorFlow Lite

Abstract

This report documents the setup, deployment, simulation, testing, and validation of a simple convolutional neural network (CNN) trained using Google Colab. The model was developed with TensorFlow and Keras, trained on the MNIST dataset, and converted to TensorFlow Lite for edge deployment simulation. Google Colab was used as the primary development environment due to its cloud-based Jupyter Notebook interface and built-in machine learning support. The results demonstrate that lightweight models can be effectively prepared for edge deployment using cloud-based tools.

1. Introduction

Edge AI systems often require models to be optimized for limited resources while maintaining acceptable performance. The purpose of this simulation was to train a basic image classification model in Google Colab and prepare it for deployment on edge devices using TensorFlow Lite. This exercise focused on understanding the full workflow from model training to deployment readiness rather than just model accuracy.

2. Setup and Environment

The entire simulation was conducted using **Google Colab**, which provides a cloud-hosted Jupyter Notebook environment with TensorFlow preinstalled. This eliminated the need for local software installation or configuration. The MNIST dataset was loaded directly using TensorFlow's dataset utilities. Before training, the image data was normalized and reshaped to match the expected input format of the convolutional neural network.

Google Colab also provided real-time execution logs and training visualizations, which were useful for monitoring model performance throughout the training process.

3. Model Design and Deployment Process

A simple convolutional neural network was designed using Keras. The architecture included a convolutional layer for feature extraction, a max pooling layer for dimensionality reduction, and fully connected dense layers for classification. The model used the Adam optimizer and sparse categorical cross-entropy loss.

After training for five epochs in Google Colab, the model was converted into a `.tflite` file using TensorFlow Lite's converter. The converted model was then downloaded directly from Colab, making it ready for deployment or further edge device simulation.

4. Testing and Validation Results

Training results showed consistent improvement in accuracy across epochs, with stable validation performance. This indicated that the model generalized well to unseen data. The TensorFlow Lite conversion completed successfully, confirming that the trained model was compatible with edge deployment requirements.

Console logs from Google Colab confirmed successful training, validation, and model conversion steps.

(Screenshot suggestion: Colab training output, model summary, TFLite conversion confirmation)

5. Conclusion

This simulation successfully demonstrated how Google Colab can be used to develop and prepare machine learning models for edge deployment. Using a cloud-based environment simplified the workflow and reduced setup complexity. The exercise provided practical experience with TensorFlow Lite and reinforced key concepts related to edge AI and model optimization.

Reflective Journal (A02)

Using Google Colab for this simulation made the entire process much easier and more efficient. I liked that everything was already set up, which allowed me to focus on learning how the model works instead of dealing with installation issues. Seeing the training progress and validation results in real time helped me better understand how the model was learning.

Converting the model to TensorFlow Lite was the most interesting part of the exercise because it showed how models are prepared for edge deployment. It made me realize that training a model is only part of the process, and deployment considerations are just as important.

Overall, this activity helped connect what we learn in class to a real-world edge AI workflow and made the idea of deploying models on edge devices feel more approachable.

References

TensorFlow. (2024). *TensorFlow Lite documentation*. <https://www.tensorflow.org/lite>
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Untitled document - Google | x Untitled5.ipynb - Colab x L03 Deploying a Simple AI M... x + Gemini

colab.research.google.com/drive/1GkmFbYwqgtITFtj1C2uF-lfr4nd0iDir#scrollTo=O7l-Qp2R42eq

Untitled5.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

```
[11] ✓ 0s
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 128)	692,352
dense_1 (Dense)	(None, 10)	1,290

Total params: 693,962 (2.65 MB)
Trainable params: 693,962 (2.65 MB)
Non-trainable params: 0 (0.00 B)

```
[12] ✓ 0s
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

Variables Terminal

10:16 AM Python 3

Untitled document - Google | x Untitled5.ipynb - Colab x L03 Deploying a Simple AI M... x + Gemini

colab.research.google.com/drive/1GkmFbYwqgtITFtj1C2uF-lfr4nd0iDir#scrollTo=O7l-Qp2R42eq

Untitled5.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

```
[13] ✓ 3m
)

Epoch 1/5
1875/1875 — 41s 21ms/step - accuracy: 0.9072 - loss: 0.3009 - val_accuracy: 0.9796 - val_loss: 0.0659
Epoch 2/5
1875/1875 — 42s 22ms/step - accuracy: 0.9839 - loss: 0.0512 - val_accuracy: 0.9827 - val_loss: 0.0489
Epoch 3/5
1875/1875 — 40s 21ms/step - accuracy: 0.9907 - loss: 0.0294 - val_accuracy: 0.9818 - val_loss: 0.0561
Epoch 4/5
1875/1875 — 42s 22ms/step - accuracy: 0.9939 - loss: 0.0193 - val_accuracy: 0.9861 - val_loss: 0.0473
Epoch 5/5
1875/1875 — 40s 21ms/step - accuracy: 0.9962 - loss: 0.0131 - val_accuracy: 0.9853 - val_loss: 0.0495
```

```
[14] ✓ 3s
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save .tflite model file
tflite_path = "model.tflite"
with open(tflite_path, "wb") as f:
    f.write(tflite_model)

print("Saved TFLite model to:", tflite_path)
```

... Saved artifact at '/tmp/tmp844jtfjq'. The following endpoints are available:

- * Endpoint 'serve'
args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28, 1), dtype=tf.float32, name='keras_tensor')

Output Type:
TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)

Captures:
137622892595152: TensorSpec(shape=(), dtype=tf.resource, name=None)
137622892596304: TensorSpec(shape=(), dtype=tf.resource, name=None)
137622892595536: TensorSpec(shape=(), dtype=tf.resource, name=None)
137622892596688: TensorSpec(shape=(), dtype=tf.resource, name=None)
137622892596496: TensorSpec(shape=(), dtype=tf.resource, name=None)

Variables Terminal

10:16 AM Python 3